Raport: DSC PJATK Catalogue Quality Challenge

Nov 4, 2026 · @decu

Grupa nr 7 · Maciej Dec (zespół jednoosobowy) · repozytorium: github.com/dcu111/dsc-catalogue-quality

Dwie zmiany w cechach podniosły OOF ROC-AUC baseline'u z 0,708 do 0,942 przy tym samym modelu i tych samych foldach po sprzedawcach.

1. Cel, dane i walidacja

Zadanie to ranking ofert e-commerce od najbardziej podejrzanej: dla każdej oferty model podaje prawdopodobieństwo, że zawiera jeden kontrolowany błąd (kategoria, cena, waga albo liczba sztuk).

Zbiór	Oferty	Uwagi
train	16 000	17,0% z błędem (target = 1); tytuł, opis, kategoria, marka, stan, cena, waga, pack_size
test	10 000	target ukryty; inni sprzedawcy niż w train

Braki danych są niewielkie (waga 6,0%, marka 5,9%, opis 4,2%, cena 3,9%) i według regulaminu same nie oznaczają błędu.

Metryka. ROC-AUC mówi, jak często losowa zła oferta dostaje wyższe prawdopodobieństwo niż losowa dobra: 0,5 to ranking losowy, 1,0 to idealny. Pomocniczo podaję Average Precision (AP), które lepiej widzi klasę mniejszościową; dla losowego modelu AP ≈ 0,17.

Walidacja. Używam podziału z baseline'u: 5 foldów rozłącznych po sprzedawcach. Numer foldu to SHA-256(„42:seller_id") modulo 5, więc ten sam sprzedawca zawsze trafia do jednej grupy, a podział nie zależy od wersji bibliotek ani od targetu. W teście są wyłącznie nowi sprzedawcy; gdyby sprzedawca był jednocześnie w nauce i w walidacji, model mógłby nauczyć się jego stylu i wynik byłby zawyżony.

Każda oferta z train dostaje predykcję od modelu, który jej nie widział (out-of-fold, OOF). Decyzje podejmuję na podstawie OOF ROC-AUC z 16 000 ofert, a nie wyniku publicznego, który obejmuje tylko 30% testu.

2. Punkt wyjścia: baseline

Baseline organizatorów osiąga OOF ROC-AUC 0,7082 i AP 0,5706 (wynik publiczny 0,69931). Wszystkie moje zmiany dotyczą wyłącznie funkcji `features()` i listy cech w `make_model()`; model, hiperparametry i foldy pozostają bez zmian.

Etap	Co robi
`features()`	Zamienia ofertę na cechy bez uczenia się: tekst tytułu i opisu, długości, logarytm ceny, wagi i pack_size, cena i waga na sztukę, liczba sztuk z tekstu (`text_pack`) i jej różnica z pack_size, pary kategoria×słowo
`make_model()`	`ColumnTransformer`: TF-IDF na tekście (1–2-gramy, 18 000 cech) i na parach kategoria×słowo (12 000), cechy liczbowe (imputacja medianą z flagą braku i skalowanie), cechy kategoryczne (one-hot); potem regresja logistyczna (C = 2, liblinear, seed 42)
`run()`	Dla każdego z 5 foldów buduje nowy model, uczy go na 4 foldach, przewiduje fold walidacyjny (OOF) i test; predykcje testu uśrednia z 5 modeli

Wszystko, co się uczy (słownik TF-IDF, mediany do imputacji, skalowanie), jest w `Pipeline`, więc liczy się tylko na foldzie treningowym. To zasada, której trzymałem się też przy własnych zmianach.

Komentarz organizatorów nad odczytem `text_pack` ostrzega, że to tylko „starter hint”, który mylą darmowe akcesoria i liczby zapisane słowami. Od sprawdzenia tej wskazówki zacząłem eksplorację.

3. Eksploracja danych

Wskazówka „liczba sztuk z tekstu” w baseline jest błędna albo pusta w około 40% ofert. Z tego wyrosło pierwsze porównanie.

Liczba sztuk z tekstu

Baseline bierze pierwszą liczbę przed słowem „pcs” (`(\d+)\s*pcs`) i porównuje ją z `pack_size`.

Obserwacja	Liczba ofert
Opisy z „N pcs”	5 735
...z czego z darmowymi akcesoriami („Also includes 3 pcs of free accessory pieces”)	3 865
Liczba przed „pcs” różna od pack_size	2 587
...z czego w opisach z gratisami	2 537 (98%)
Opisy z liczbą zapisaną słowem (one–five)	5 265
...z czego bez „pcs”, czyli baseline nic nie odczyta	3 979

Wniosek: w ofertach z gratisami baseline czyta liczbę gratisów zamiast głównych sztuk, a liczb zapisanych słowem nie widzi wcale. Alarm „liczba się nie zgadza” piszczy więc głównie w dobrych ofertach.

Jakie formuły ilości występują. Podzieliłem opisy na zdania i zamieniłem każdą liczbę (także słownie) na `#`. Wtedy zdania z różnymi liczbami wyglądają tak samo, a `value_counts` pokazuje listę szablonów. Wyszło 7 formuł ilości (np. „pack of # main items”, „bundle quantity: # units”, „sold as a #-pack”) oraz zdanie o gratisach. Jedyna pułapka to „#-string small guitar”, gdzie liczba oznacza liczbę strun.

Hipoteza odrzucona: marka w tytule

Sprawdziłem, czy brak marki w tytule (np. literówka „Velo” zamiast „Velto”) wskazuje złą ofertę. Takich ofert było 145 i 20,0% z nich było złych, przy 17,1% w pozostałych. Przy tak małej grupie i różnicy to za słaby sygnał, więc nie zrobiłem z tego cechy. Literówki wyglądają raczej na szum dodany do wszystkich ofert.

4. Porównanie 1: odczyt liczby sztuk

Nowy odczyt liczby głównych sztuk podniósł OOF ROC-AUC z 0,7082 do 0,7324 (+0,024), a wszystkie 5 foldów wzrosło.

Hipoteza. Jeśli model dostanie prawdziwą liczbę głównych sztuk z tekstu, lepiej wykryje oferty z niezgodnym `pack_size`.

Zmiana (jedna grupa cech). Kolumnę `text_pack` liczy teraz funkcja `main_items` zamiast `(\d+)\s*pcs`. Reszta modelu, hiperparametry i foldy są bez zmian.

```
WORDS = {'one': '1', 'two': '2', 'three': '3', 'four': '4', 'five': '5'}
QTY = (r'(?:pack of|contains|main items:|quantity:?)\s*(\d+)'
       r'|(\d+)\s*(?:-pack|units|pcs|main items|individually)')

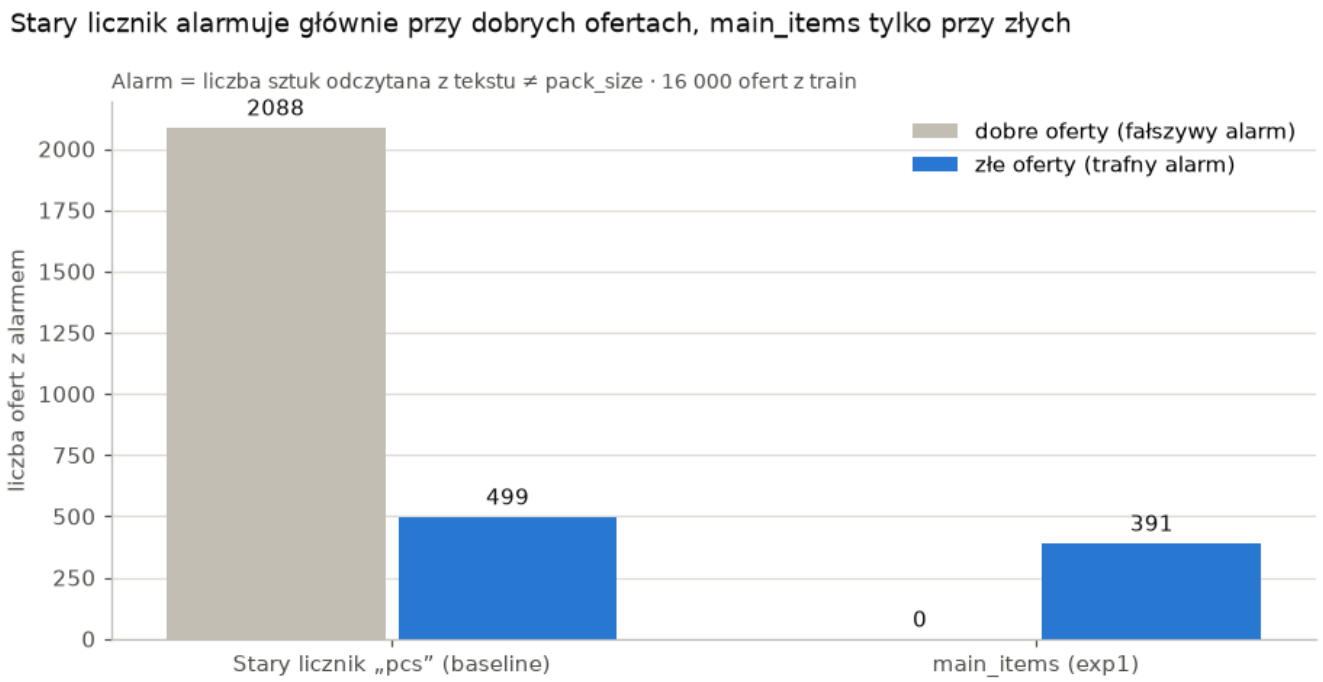
def main_items(text):
    text = text.lower()
    text = re.sub(r'also includes \w+ pcs of free accessory pieces', ' ',
text)
    for word, digit in WORDS.items():
        text = re.sub(r'\b' + word + r'\b', digit, text)
    found = re.search(QTY, text)
    if found is None:
        return np.nan
    return float(found.group(1) or found.group(2))
```

Funkcja usuwa zdanie o gratisach, zamienia liczby słowne na cyfry i szuka liczby przy 7 formułach z eksploracji. Kolejność jest ważna: gdyby gratisy nie zostały usunięte najpierw, wzorzec „# pcs" złapałby ich liczbę. Funkcja czyta tylko tytuł i opis i niczego się nie uczy, więc nie wprowadza przecieku.

Sprawdzenie przed modelem. Odczyt działa w 15 951 z 16 000 ofert. Różnica z `pack_size` pojawia się w 0 dobrych i 391 złych ofertach; stary odczyt dawał 2 088 fałszywych alarmów w dobrych ofertach i 499 w złych.

	Baseline	exp1	Różnica
OOF ROC-AUC	0,7082	**0,7324**	**+0,0243**
Fold 1	0,6983	0,7306	+0,0323
Fold 2	0,6810	0,7107	+0,0298
Fold 3	0,7184	0,7408	+0,0224
Fold 4	0,7167	0,7382	+0,0215
Fold 5	0,7226	0,7393	+0,0166
Average Precision	0,5706	0,6237	+0,0531
Czas [s]	22,4	24,4	+2,0
Wynik publiczny	0,69931	0,71488	+0,0156

Wzrost we wszystkich foldach (od +0,017 do +0,032) wskazuje, że to nie przypadek jednego podziału. Wynik publiczny podaję tylko informacyjnie; nie służył do wyboru metody. Nowy odczyt zostaje w modelu i jest punktem odniesienia dla porównania 2.



Wykres 1. Stary licznik alarmuje w 2088 dobrych ofertach, main_items tylko w złych (391). Źródło: notebook EDA, 16 000 ofert z train.

5. Porównanie 2: cena i waga względem produktu

Odległość ceny i wagi za sztukę od typowej wartości dla tego samego produktu podniosła OOF ROC-AUC z 0,7324 (exp1) do 0,9419 (+0,209). Wszystkie 5 foldów wzrosło o 0,20–0,23.

Skąd pomysł. Pierwsza oferta w train to statyw mikrofonowy, który waży 5,233 kg i jest zły, podczas gdy typowy statyw waży około 1,9 kg. Baseline ma cenę i wagę na sztukę, ale regresja logistyczna daje każdej kolumnie jedną wagę, czyli może wyrazić tylko „więcej znaczy bardziej podejrzane". Tymczasem 5 kg to dużo jak na statyw i mało jak na uchwyt do monitora.

Hipoteza. Oferta jest podejrzana, gdy jej cena albo waga za sztukę mocno odstaje od typowej dla tego samego produktu.

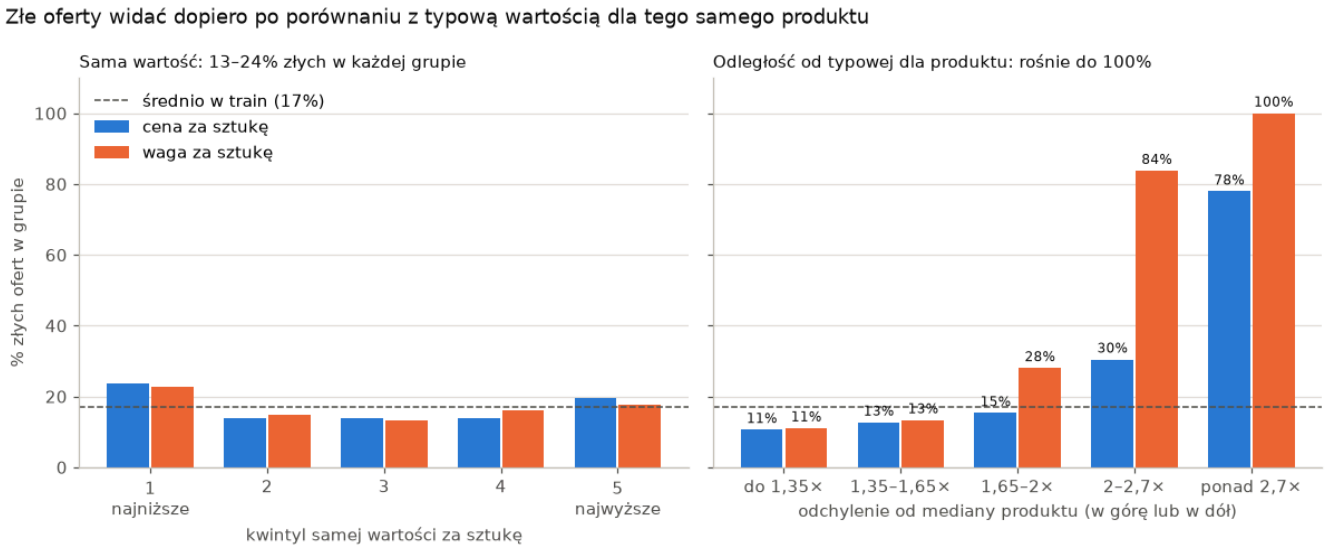
Sprawdzenie w eksploracji

Nazwę produktu odczytuję z opisu, który zawsze zawiera zdanie „<produkt> by <marka>" (regex `(?:^|\.\s*)([a-z][a-z \-]+?) by \w+`). Tytuły używają synonimów, a opis nie. Produkt jest rozpoznany w 15 320 z 16 000 ofert (pozostałe nie mają opisu); różnych produktów jest 180.

Odległość to |log(wartość na sztukę) − mediana log dla produktu|. Logarytm sprawia, że liczy się „ile razy", a nie „o ile". Mediana jest odporna na skrajne wartości, które często są właśnie błędnymi ofertami.

Odległość od typowej	Waga: ofert	Waga: złych	Cena: ofert	Cena: złych
0–0,3	10 198	11,1%	7 511	10,6%
0,3–0,5	2 308	13,3%	3 208	12,7%
0,5–0,7	638	28,1%	1 702	15,3%
0,7–1,0 (ok. 2–2,7×)	259	83,8%	1 014	30,5%
ponad 1,0 (ponad 2,7×)	539	100%	825	77,9%
AUC samej wartości na sztukę		0,468		0,473
AUC odległości od typowej		0,692		0,697

Sama waga i cena na sztukę są prawie losowe (AUC ok. 0,47), a odległość od typowej daje około 0,70. Dziwna waga oznacza złą ofertę częściej niż dziwna cena, bo ceny tego samego produktu naturalnie się różnią (promocje, stan używany). W eksploracji mediany liczyłem z całego train; w modelu tak nie wolno.



Wykres 2. Sama cena i waga za sztukę: 13–24% złych ofert w każdej grupie; odległość od mediany produktu: do 78% (cena) i 100% (waga). Źródło: notebook EDA (mediany z całego train, tylko eksploracja).

Zmiana w modelu bez przecieku

Mediany są czymś wyuczonym, więc muszą się liczyć tylko na foldzie treningowym. Dlatego zamknąłem je we własnym transformerze sklearn, który działa w `Pipeline`:

```
class OffTypical(BaseEstimator, TransformerMixin):
    def per_item(self, X):
        pack = X.pack_size.clip(lower=1)
        return pd.DataFrame({'product': X['product'],
                             'price': np.log(X.price_pln / pack),
                             'mass': np.log(X.shipping_weight_kg / pack)})

    def fit(self, X, y=None):
        self.typical_ = self.per_item(X).groupby('product').median()
        return self

    def transform(self, X):
        items = self.per_item(X)
        typical = self.typical_.reindex(items['product']).to_numpy()
        return np.abs(items[['price', 'mass']].to_numpy() - typical)
```

W `features()` dochodzi tylko kolumna `product` (odczyt tekstu, bez uczenia). W `make_model()` dochodzi taśma `OffTypical → SimpleImputer(median, add_indicator) → StandardScaler`, taka sama jak dla pozostałych cech liczbowych. Produkt nieznany w foldzie treningowym daje NaN, który imputer zamienia na medianę z flagą braku.

`BaseEstimator` pozwala sklearn klonować transformer w każdym foldzie (kopiowane są ustawienia, nie nauczone mediany), a `TransformerMixin` daje `fit_transform`. Na foldzie walidacyjnym i na teście wołany jest tylko `transform` z medianami z treningu.

Wynik

	exp1	exp2	Różnica
OOF ROC-AUC	0,7324	**0,9419**	**+0,2094**
Fold 1	0,7306	0,9407	+0,2101
Fold 2	0,7107	0,9387	+0,2280
Fold 3	0,7408	0,9474	+0,2066
Fold 4	0,7382	0,9435	+0,2053
Fold 5	0,7393	0,9390	+0,1997
Average Precision	0,6237	0,8717	+0,2480
Odchylenie std. foldów	0,0112	0,0032	−0,0080
Czas [s]	24,4	22,4	−2,0

Dlaczego to nie przeciek, choć skok jest duży

1. Cecha nie używa targetu: `fit` przyjmuje `y`, ale go nie czyta.
2. Mediany powstają w `fit`, czyli tylko z ofert treningowych danego foldu; walidacja i test przechodzą tylko przez `transform`.
3. Wzrost jest równy we wszystkich foldach, a rozrzut foldów zmalał.
4. Już w eksploracji sama odległość dawała AUC około 0,70 dla każdej z dwóch cech, więc duży wzrost ma wytłumaczenie w danych.
5. Lokalne sprawdzenie: mediany w transformerze po `fit` na foldzie 1 są identyczne z medianami policzonymi ręcznie tylko z ofert spoza tego foldu.

6. Analiza 5 błędów OOF

Przy budżecie przeglądu 17% model exp2 trafia w 78,9% alarmów i łapie 78,9% złych ofert. Pięć najpewniejszych pomyłek wskazuje jeden ślepy punkt: typowa wartość miesza warianty produktu (large, standard, compact) i stany (new, used).

Reguła decyzji

Moderator sprawdza tyle ofert, ile jest złych w train, czyli 2 721 (17%), zaczynając od najwyższego prawdopodobieństwa. Daje to próg $p \geqslant 0{,}2883$. Reguła ma uzasadnienie biznesowe (ograniczony czas moderacji), odpowiada precision@k i jest wyznaczona wyłącznie z predykcji OOF na train.

	Decyzja 0: przepuść	Decyzja 1: sprawdź
Prawda 0: dobra	12 706 (TN)	573 (FP)
Prawda 1: zła	573 (FN)	2 148 (TP)

Precision = recall = 2 148 / 2 721 = 78,9%. FP równa się FN, bo budżet jest równy liczbie złych ofert: każdy fałszywy alarm zajmuje miejsce jednej złej oferty.

Zasada wyboru przypadków

Wybrałem 3 fałszywe alarmy z najwyższym p i 2 przegapione złe oferty z najniższym p. Są to pomyłki, przy których model był najbardziej pewny, więc najlepiej pokazują systematyczne braki; pomyłki tuż przy progu często wynikają z przypadku. Zasada nie używa losowania, więc wybór jest odtwarzalny.

Dla każdego przypadku porównałem cenę i wagę za sztukę z medianą policzoną **tylko z innych foldów**, czyli z tego, co model widział, zanim ocenił ofertę: dla tego samego produktu, a potem dodatkowo dla tego samego wariantu (z opisu „Variant: …”) i stanu.

Przypadki

ID	Fold	Prawda	p	Decyzja	Fakty	Możliwe wyjaśnienie (niepewne)	Co sprawdzić
OFR_710938274	2	0	0,987	1 (FP)	Piano foot controller, large, new, bez tytułu. Cena 224,44 zł przy medianie produktu 49,17 zł (4,6×, 11 ofert); dla tego samego wariantu i stanu jest tylko 1 oferta. Waga 0,886 kg, blisko mediany dla large (0,815 kg).	Model zareagował na cenę. Produkt jest rzadki, więc mediana z 11 ofert może nie oddawać zakresu cen; oferta może być droższym modelem. Powodu wysokiej ceny nie znamy.	Przy rzadkich produktach mniej ufać medianie albo porównywać z kategorią; policzyć, jaka część FP dotyczy rzadkich produktów.
OFR_219119666	1	0	0,940	1 (FP)	Illustrated vocabulary book, large, new, bez tytułu. Cena 208,22 zł: 3,7× mediany produktu (63 oferty), 2,4× mediany dla large i new (86,74 zł, 8 ofert). Waga 1,605 kg: 2,0× produktu, 1,3× dla large.	Część różnicy tłumaczy wariant large, którego mediana nie uwzględnia; po jego uwzględnieniu waga jest prawie typowa. Cena nadal jest wysoka z nieznanego powodu.	Mediana dla produktu i wariantu; kontrolowane porównanie z exp2.

ID	Fold	Prawda	p	Decyzja	Fakty	Możliwe wyjaśnienie (niepewne)	Co sprawdzić
OFR_932756223	2	0	0,939	1 (FP)	Recipe book, large, new, 2 szt. i 3 darmowe akcesoria. Cena za sztukę 187,33 zł: 3,6× produktu, 2,5× dla large i new (75,21 zł). Waga za sztukę 1,367 kg: 2,1× produktu, 1,5× dla large i new.	Wariant large tłumaczy część różnicy. Waga paczki może obejmować 3 gratisy, a dzielę ją tylko przez 2 główne sztuki.	Porównać wagę ofert z gratisami i bez dla tego samego produktu.
OFR_447956039	5	1	0,001	0 (FN)	Insulated lunch bag, compact, used, 3 szt. i 2 gratisy, „Clearance promotion” w opisie, literówka „tree” w tytule. Cena za sztukę 44,97 zł: 0,75× produktu, ale 1,5× dla compact i used (30,24 zł, 4 oferty). Waga 1,2× dla compact i used.	Na tle wszystkich lunch bagów oferta wygląda zwyczajnie, więc model nic nie widzi. Na tle małych używanych, do tego na wyprzedaży, jest droższa. Grupa ma tylko 4 oferty; nie wiemy, co jest błędne.	Mediana dla produktu, wariantu i stanu; sprawdzić, czy dobre oferty z „Clearance promotion” są tańsze.

ID	Fold	Prawda	p	Decyzja	Fakty	Możliwe wyjaśnienie (niepewne)	Co sprawdzić
OFR_362706930	5	1	0,003	0 (FN)	USB video webcam, compact, used. Cena 144,48 zł: 1,2× mediany produktu (119,07 zł, 100 ofert), ale 2,0× mediany dla compact i used (73,05 zł, 6 ofert). Waga 1,3× dla compact i used.	Model porównuje z wszystkimi kamerkami, gdzie cena jest zwyczajna; na tle małych używanych jest 2× za wysoka. Może to być powód, ale ukrytego typu błędu nie znamy.	Mediana dla produktu i wariantu (i stanu); sprawdzić, czy ta oferta dostaje wtedy wyższe p.

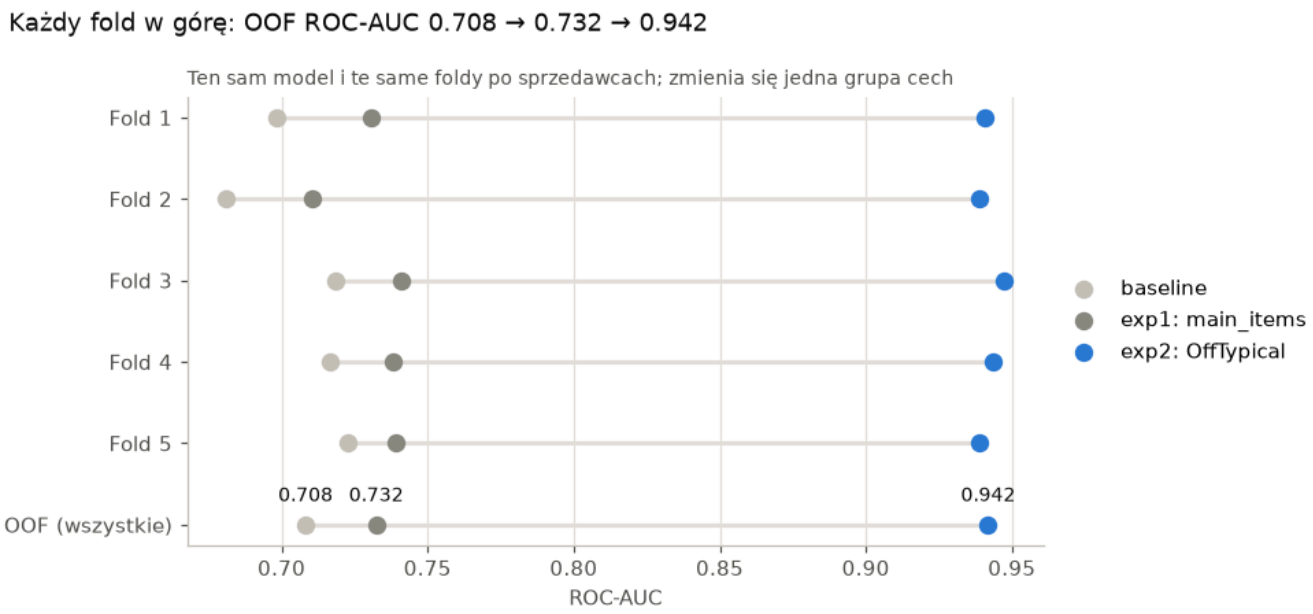
Wspólny wniosek

- **Fakt:** wszystkie 3 fałszywe alarmy to wariant large, oba przegapione to compact i used. Przy porównaniu z właściwą grupą różnica dla FP maleje (np. 3,7× → 2,4×), a dla FN rośnie (np. 1,2× → 2,0×).
- **Niepewne:** dlaczego te konkretne oferty są dobre albo złe; grupy porównawcze bywają małe (1–10 ofert). Nie twierdzę, że znam ukryty typ błędu żadnej oferty.
- **Propozycja:** w `OffTypical` liczyć medianę dla produktu i wariantu zamiast samego produktu, z kontrolowanym porównaniem względem exp2.

7. Wnioski i dalsze kroki

Największy zysk dało porównywanie wartości z właściwym punktem odniesienia, a nie zmiana modelu. Model liniowy nie umie sam stwierdzić, że „drogo” znaczy co innego dla strun gitarowych i dla uchwytu do monitora; trzeba mu podać cechę, która już to uwzględnia.

Wersja	OOF ROC-AUC	AP	Wynik publiczny
baseline	0,7082	0,5706	0,69931
exp1: `main_items`	0,7324	0,6237	0,71488
exp2: `OffTypical`	0,9419	0,8717	0,94576



Wykres 3. Każdy z 5 foldów rośnie przy obu zmianach. Źródło: notebook modelu.

Ograniczenia.

- Mediana dla rzadkich produktów (np. 11 ofert) jest niepewna.
- Typowa wartość nie uwzględnia wariantu, stanu ani promocji, co widać w analizie błędów.
- Błędy kategorii wykrywają tylko cechy z baseline'u (pary kategoria×słowo); nie dodawałem dla nich osobnej cechy.

Dalsze kroki (każdy jako osobne porównanie kontrolowane):

1. Mediana dla produktu i wariantu w `OffTypical`, z powrotem do samego produktu przy małych grupach.
2. Uwzględnienie stanu (used, refurbished) i zdania „Clearance promotion" przy typowej cenie.
3. Zgodność kategorii: jak często dany produkt w foldzie treningowym ma zadeklarowaną kategorię tej oferty.

8. Największe wyzwania

- **Praca w pojedynkę, od podstaw.** Druga osoba z zespołu przestała odpowiadać, więc całość zrobiłem sam. Dopiero zaczynam naukę data science, więc najwięcej czasu zajęło mi zrozumienie kodu baseline'u (`Pipeline` , `ColumnTransformer` , TF-IDF, walidacja po sprzedawcach). W nauce i pisaniu kodu mocno pomagał mi asystent AI (sekcja 10).

- **Walidacja bez przecieku.** Mediany produktu są czymś wyuczonym, więc nie wolno ich liczyć na całym zbiorze. Rozwiązanie: własny transformer sklearn `OffTypical` w `Pipeline`, który liczy mediany w `fit`, czyli tylko na foldzie treningowym. Musiałem też sprawdzić, że skok z 0,73 do 0,94 nie jest przeciekiem (sekcja 5).
- **Liczba sztuk w wolnym tekście.** Opisy mają wiele szablonów, gratisy i liczby zapisane słowem. Szablony znalazłem, zamieniając liczby na `#` i licząc powtórzenia; potem napisałem wzorzec `QTY` i funkcję `main_items`.
- **Cena bez punktu odniesienia.** Sama cena i waga za sztukę mają AUC ok. 0,47, bo „drogo” zależy od produktu. Trzeba było znaleźć, z czym je porównać: z typową wartością dla tego samego produktu, którego nazwę wyciągam z opisu.
- **Reguła decyzji bez etykiet testu.** Analiza błędów wymaga progu; wybrałem budżet przeglądu 17%, wyznaczony tylko z predykcji OOF na train.
- **Kaggle w praktyce.** Szybki zapis notebooka nie uruchamia kodu, więc pierwsze zgłoszenie miało wynik exp1 zamiast exp2; po imporcie notebooka trzeba też ponownie dołączyć dane konkursu. Teraz przed wysłaniem sprawdzam wersję notebooka i jego wynik.

9. Odtwarzalność

Całe rozwiązanie liczy się na CPU w około 25 sekund, bez internetu i bez zewnętrznych danych czy modeli.

Element	Wartość
Środowisko	Kaggle Notebook, CPU
Wersje	Python 3.13, numpy 2.1.3, pandas 2.3.3, scikit-learn 1.6.1
Seed	42 (regresja logistyczna i podział na foldy)
Foldy	SHA-256(„42:seller_id”), pierwsze 16 znaków szesnastkowych modulo 5
Czas	22,4 s (exp2)
Pliki wynikowe	`submission.csv`, `oof.csv`, `cv_report.json`

Repozytorium. github.com/dcu111/dsc-catalogue-quality. Zawiera dwa notebooki: `dsc-catalogue-model.ipynb` (model, walidacja, plik zgłoszenia i analiza błędów) oraz `dsc-catalogue-eda.ipynb` (eksploracja), a także README i `requirements.txt`. Danych konkursu w repozytorium nie ma.

Jak uruchomić. Na Kaggle: zaimportować notebook, dołączyć dane konkursu i uruchomić wszystkie komórki. Lokalnie: pliki CSV w `data/public/`. Notebook EDA nie wpływa na model; target służy w nim tylko do sprawdzania hipotez na train.

Oba notebooki są zapisane w repozytorium razem z wynikami i wykresami, więc można je przeczytać bez uruchamiania.

10. Użycie AI

Korzystałem z asystenta AI (Claude, Anthropic, w aplikacji Claude Code) do tłumaczenia kodu i regulaminu, proponowania pomysłów i pisania kodu. AI nie dostarczało etykiet, predykcji, embeddingów ani cech dla pojedynczych ofert; wszystkie cechy liczy kod z danych konkursu.

Dopiero zaczynam naukę data science, więc pomoc AI była duża: bez niej nie zbudowałbym jeszcze samodzielnie całego rozwiązania. Każdy krok uruchamiałem sam, przeszedłem przez kod funkcja po funkcji i rozumiem, co i dlaczego robi każda zmiana. Naukę tych tematów kontynuuję (kursy Kaggle Learn).

Etap	Co zrobiło AI	Co zrobiłem sam i jak sprawdziłem
Baseline	Wytłumaczyło kod i regulamin	Uruchomiłem baseline na Kaggle; wynik zgodny z opisem (0,7082)
Eksploracja	Zaproponowało komórki sprawdzające „pcs”, gratisy i liczby słowne oraz trik z `#`; policzyło hipotezę o marce	Uruchomiłem komórki, porównałem liczby, zapisałem obserwacje w dzienniku
Porównanie 1	Napisało `main_items` i test na przykładowych zdaniach	Sprawdziłem test i crosstab z targetem, uruchomiłem model; wynik równy lokalnemu sprawdzeniu AI
Porównanie 2	Zaproponowało porównanie z medianą produktu i klasę `OffTypical`	Sprawdziłem trop w eksploracji (AUC 0,69 i 0,70), uruchomiłem model, przeanalizowałem ryzyko przecieku
Analiza błędów	Zaproponowało regułę budżetu i zasadę wyboru, napisało kod porównań	Sam zauważyłem, że fałszywe alarmy to drogie oferty; potwierdziłem wariant liczbami
Raport i README	Przygotowało szkic na podstawie dziennika eksperymentów	Przeczytałem, poprawiłem i uzupełniłem

AI uruchamiało kod najpierw lokalnie na kopii danych, żeby sprawdzić poprawność i podać oczekiwany wynik. Każdą komórkę uruchamiałem potem sam na Kaggle i porównywałem liczby; wszystkie wyniki się zgadzały. Pełny dziennik użycia AI prowadziłem w arkuszu „Użycie AI”.

Kod bazowy: `baseline_solution.ipynb` organizatorów konkursu.